

STM32 Offline Text-to-Speech System

Project Overview

A complete **offline text-to-speech (TTS)** system for STM32 microcontrollers using pre-recorded natural human voice. No internet required, no external chips needed - everything runs on your STM32!

What It Does

- Speaks English text using natural human voice
- Completely offline - no WiFi, no cloud services
- Uses compressed audio to fit in limited STM32 flash memory
- Automatically pronounces unknown words phonetically
- Plays long words using smart chunking (no buffer overflow)

Demo

STM32 says: "hello world, system is working"

Output: "hello w-er-l-d system is working"

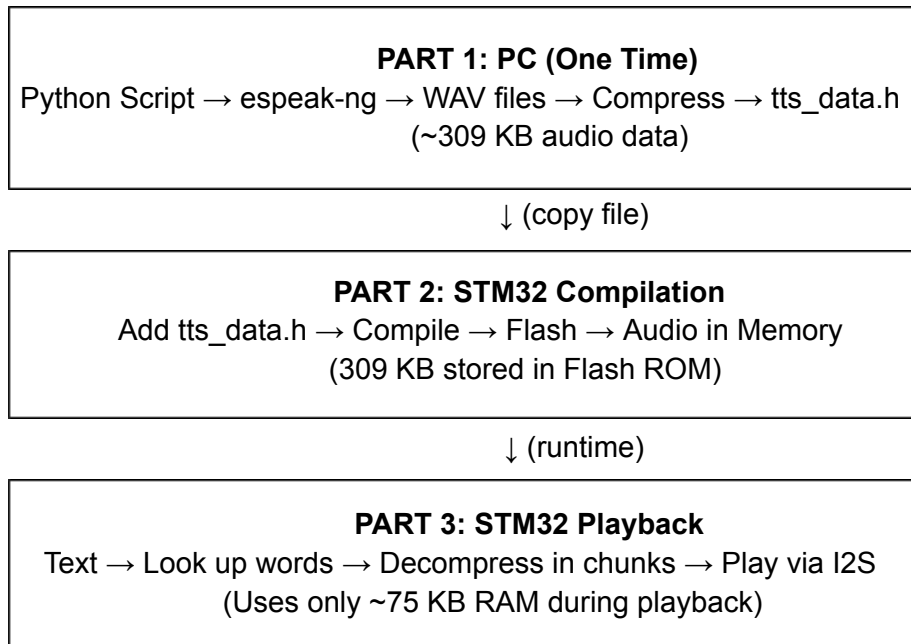
(Unknown words pronounced phonetically)

How It Works

The Big Picture

1. **Dictionary Creation (PC):** Record 50+ words, compress them, save to a C header file
2. **Store in STM32:** Upload the dictionary to STM32's permanent storage (flash memory)
3. **Playback:** When STM32 wants to speak "hello world", it:
 - Finds "hello" in dictionary → plays clear recording
 - Can't find "world" → pronounces it phonetically: "w-er-l-d"
 - Uses chunked playback for long words (no memory issues)

Three-Part System



STM32 Memory

1. Flash Memory (ROM)

- **Size:** 512 KB
- **Type:** Read-Only Memory (permanent storage)
- **Speed:** Slow to write, fast to read
- **What's stored here:**
 - Compiled program code (~50 KB)
 - Audio data from tts_data.h (~300 KB)
 - Constants and lookup tables
- **Can be erased:** Only by re-flashing the chip

Example in code:

```
const uint8_t tts_word_hello[] = {0x45, 0x23, ...}; // Stored in Flash
```

2. RAM (SRAM)

- **Size:** 128 KB
- **Type:** Random Access Memory (temporary workspace)
- **Speed:** Very fast read/write

- **What's stored here:**
 - Variables while program is running
 - Audio buffers during playback
 - Stack (function calls, local variables)
 - Heap (malloc/free memory)
- Everything erased when power off

Example in code:

```
int16_t playbackBuffer[17640]; // Uses 35 KB of RAM
int counter = 0;               // Uses 4 bytes of RAM
```

3. Registers

- **Size:** Tiny (few dozen bytes)
 - **Type:** CPU registers (fastest memory)
 - **Speed:** Instant access (inside CPU)
 - **What's stored here:**
 - Currently executing instructions
 - Temporary calculation results
 - Compiler manages it automatically
-

Hardware Requirements

Minimum Setup

- **MCU:** STM32F411RE (or any STM32 with $\geq 512\text{KB}$ Flash, $\geq 128\text{KB}$ RAM)
- **Audio Codec:** ES8388 (I2S audio chip)
- **Speaker:** Any 4-8 Ω speaker or headphones
- **Connections:**
 - I2C1: ES8388 control (configure codec settings)
 - I2S3: ES8388 audio data (send audio samples)
 - UART2: Debug output (optional, for monitoring)

Pinout (STM32F411RE)

STM32 Pin	Function	Connects To
PB6	I2C1_SCL	ES8388 SCL
PB7	I2C1_SDA	ES8388 SDA

PC7	I2S3_MCK	ES8388 MCLK
PC10	I2S3_SCK	ES8388 SCLK
PC12	I2S3_SD	ES8388 SDIN
PA4	I2S3_WS	ES8388 LRCK

Software Requirements

On PC

1. Python 3.7+ (for audio generation)
2. espeak-ng (text-to-speech engine)
 - Windows: [Download installer](#)
 - Linux: `sudo apt install espeak-ng`
 - Mac: `brew install espeak-ng`

Python Libraries:

`pip install numpy`

On STM32

1. STM32CubeIDE
 2. STM32 HAL Library
 3. Project Files
 - `main.c` - Main control logic
 - `tts_data.h` - Generated audio data
 - `es8388.c/.h` - Audio codec driver
 - Standard STM32 peripheral files (I2C, I2S, DMA, UART)
-

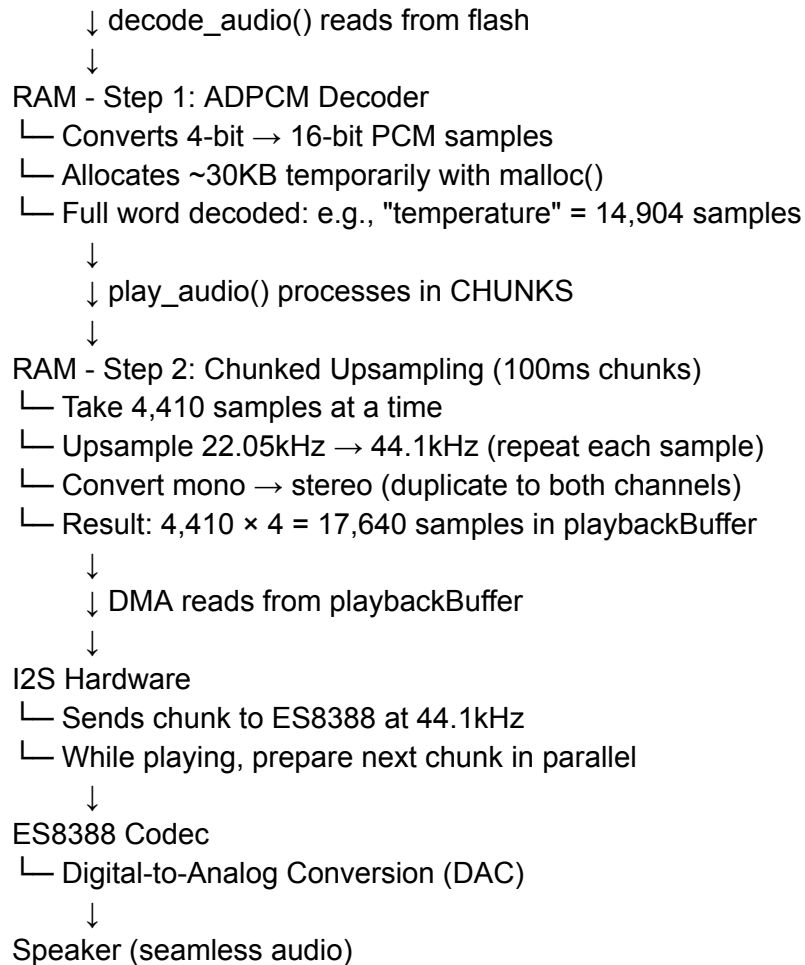
How Audio Processing Works

Complete Audio Pipeline

FLASH Memory (Permanent Storage)

- └ `tts_data.h` contains:
 - └ Compressed ADPCM audio (4-bit per sample)
 - └ 41 words + 26 letters = ~309 KB

↓



Chunking Process (Visual)

Full Word in RAM (decoded):

 14,904 samples

Split into chunks:

Chunk 1:  4,410 samples → Upsample → Play

Chunk 2:  4,410 samples → Upsample → Play

Chunk 3:  4,410 samples → Upsample → Play

Chunk 4:  1,674 samples → Upsample → Play

Each chunk reuses the same 35KB playbackBuffer!

Current Vocabulary (41 Words + 26 Letters)

System Words (20)

hello, ready, error, warning, system, sensor, temperature, working, pressure, battery, degrees, celsius, percent, high, low, is, and, the, on, off

Numbers (21)

zero, one, two, three, four, five, six, seven, eight, nine, ten, eleven, twelve, thirteen, fourteen, fifteen, sixteen, seventeen, eighteen, nineteen, twenty

Letters (26)

A-Z (pronounced phonetically: "ay", "bee", "see", etc.)

Performance Specs

Metric	Value
Vocabulary	41 words + 26 letters
Audio Quality	44.10 kHz, 16-bit
Compression Ratio	4:1 (ADPCM)
Flash Usage	300 KB audio + 50 KB code = 350 KB (70%)
RAM Usage (Peak)	80 KB (62%)
RAM Usage (Idle)	45 KB (35%)
Playback Latency	<50ms (word lookup + first chunk)
Chunk Size	100ms (4,410 samples)
Buffer Reuse	Yes - same 35KB for all chunks
CPU Usage	<5% during playback (DMA handles transfer)
Max Word Length	Unlimited

Future Enhancements

1. **Better phonetic engine:** Combine letters into syllables for smoother pronunciation
 2. **Dynamic vocabulary loading:** Store extra words on SD card, load on demand
 3. **Pitch/speed control:** Real-time audio DSP effects
 4. **Number-to-speech:** "123" → "one hundred twenty three" (instead of "one two three")
 5. **Multi-language support:** Load different espeak voices
-

Summary

This system gives us **human-quality offline speech** on a microcontroller with **smart memory management**:

- Natural voice
- No internet needed
- Low memory usage (chunked playback)
- Unlimited word length support
- Easy to customize
- Automatic phonetic fallback